

HỆ THỐNG PHÁT HIỆN MÃ ĐỘC CHUỖI CUNG ỨNG PYTHON DỰA TRÊN PHÂN TÍCH TÌNH ĐA TẦNG KẾT HỢP KHẢ NĂNG SUY LUẬN CỦA MÔ HÌNH NGÔN NGỮ LỚN (LLM)

Bùi Phú Thanh Hiền - 250202036

Tóm tắt

- Lớp: CS2205.RM
- Link Github của nhóm:
<https://github.com/hienthanh1202/CS2205.RM>
- Link YouTube video: <https://youtu.be/gXf4yUMoBfU>



Họ và Tên: Bùi Phú Thanh Hiền

MSSV: 250202036

Giới thiệu

Bối cảnh:

- Sự gia tăng đột biến của các cuộc tấn công chuỗi cung ứng Python
- Phương pháp phát hiện truyền thống dựa trên luật (Rule-based) có tỷ lệ cảnh báo giả (False Positives) cao
- Khả năng nhận diện kém đối với các kỹ thuật làm rối mã (Obfuscation) tinh vi

Động lực:

- Việc xây dựng một cơ chế thẩm định tự động có khả năng "hiểu" được ý đồ của mã nguồn là yếu tố then chốt để bảo vệ hạ tầng CI/CD của doanh nghiệp

Mục tiêu

Mục tiêu:

- Phát triển hệ thống lai (Hybrid System) giúp lọc 80% mã lành tính ở tầng tĩnh và phân tích sâu 20% mã nghi vấn ở tầng AI.
- Xây dựng bộ quy tắc "Expert Prompting" để bóc tách mã làm rối (Obfuscation) với độ chính xác cao.
- Đạt được chỉ số F1-Score vượt trội so với các công cụ Rule-based truyền thống trên cùng một tập dữ liệu thử nghiệm.

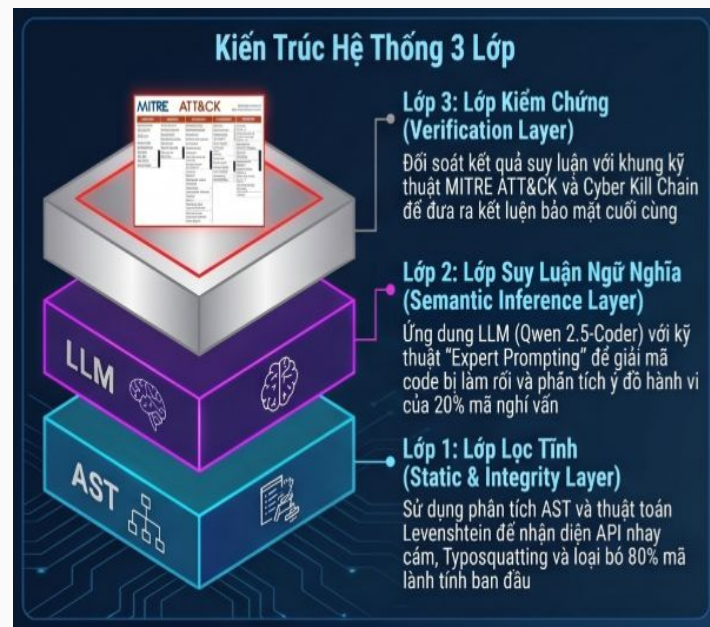
Nội dung và Phương pháp

Phương pháp nghiên cứu: Framework đề xuất kiến trúc ba lớp tầng

(1) **Lớp lọc tĩnh:** Trích xuất AST để nhận diện các API nhạy cảm, đồng thời áp dụng khoảng cách Levenshtein nhằm phát hiện các trường hợp typosquatting. Đây đóng vai trò như một lớp tiền xử lý giúp tối ưu chi phí vận hành AI, thông qua việc tích hợp các bộ lọc như Semgrep/CodeQL/Bandit để phát hiện sớm các đoạn mã nhạy cảm

(2) **Lớp suy luận ngữ nghĩa:** Sử dụng LLM (Ollama với Qwen 2.5-Coder 7B hoặc 14B) với prompt chuyên biệt để thực hiện "Semantic Abstraction". Tại đây, hệ thống sẽ giải mã các chuỗi làm rối và mô tả hành vi của mã dưới dạng ngôn ngữ tự nhiên

(3) **Lớp kiểm chứng:** So khớp kết quả từ LLM với các hành vi đặc trưng của malware chuỗi cung ứng (Exfiltration, Persistence, Discovery) với khung kỹ thuật MITRE ATT&CK để đưa ra kết luận cuối cùng



Nội dung và Phương pháp



Giả thuyết: Bằng cách sử dụng AST làm "phễu lọc" đầu vào, hệ thống sẽ loại bỏ được các cảnh báo nhiễu, đồng thời cung cấp đủ ngữ cảnh để LLM không bị hiện tượng "ảo giác" (hallucination) khi phân tích mã, vừa kiểm soát dữ liệu.

Nội dung và Phương pháp

Dataset: Xây dựng bộ dữ liệu thực nghiệm quy mô từ 1,300 đến 2,000 mẫu:

- **Malware Set:** 1,000–1,500 mẫu mã độc thực tế (từ Backstabber's Knife Collection, PyPIMalRegistry) bao gồm các mẫu có kỹ thuật che giấu phức tạp.
- **Benign Set:** 300–500 mẫu lành tính từ Top-downloaded PyPI để kiểm tra độ nhạy và tỷ lệ nhiễu của hệ thống.

Implementation: Sử dụng Python để viết script trích xuất đặc trưng; tích hợp thư viện GUMTREE hoặc công cụ AST chuẩn; triển khai pipeline gọi API LLM với cấu hình Expert Prompting chuyên biệt cho bảo mật.

Evaluation: Đánh giá qua ma trận nhầm lẫn (Precision, Recall, F1-Score). Thực hiện **Ablation Study** (nghiên cứu cắt bỏ) để chứng minh giá trị của việc kết hợp cả hai tầng Static và LLM.



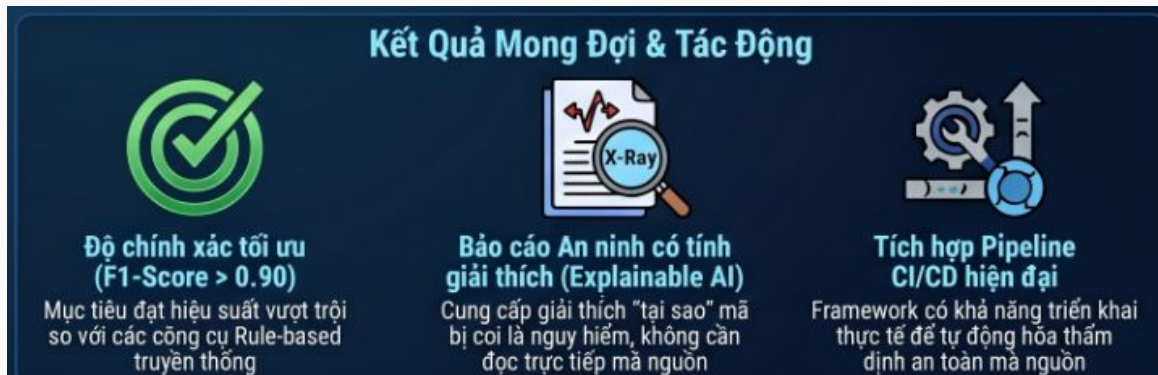
Kết quả dự kiến

Định lượng:

- Kỳ vọng giảm tỷ lệ cảnh báo giả ít nhất so với các công cụ chỉ dựa trên luật (rule-base)
- Tăng khả năng phát hiện malware

Định tính: Sản phẩm đầu ra là một báo cáo phân tích có tính giải thích (Explainable Security Report), giúp kỹ sư bảo mật hiểu rõ "tại sao" một đoạn mã bị coi là nguy hiểm mà không cần đọc trực tiếp mã nguồn phức tạp.

Tác động: Đóng góp một phương pháp luận mới trong việc ứng dụng AI vào bảo mật chuỗi cung ứng, có khả năng triển khai thực tế vào các pipeline CI/CD hiện đại.



Tài liệu tham khảo

[1]. Ridwan Shariffdeen , Behnaz Hassanshahi, Martin Mirchev:

Detecting Python Malware in the Software Supply Chain with Program Analysis

[2]. Wenbo Guo, Zhengzi Xu, Chengwei Liu, Cheng Huang, Yong Fang, Yang Liu

An Empirical Study of Malicious Code In PyPI Ecosystem